

Punto 1: Diagnóstico inicial mediante DFS

Proceso de desarrollo:

La verdad no recordaba con claridad cómo programar la lógica de búsqueda en grafo, así que en vez de pedirle a la IA el código directamente, trabajé con ella de forma guiada: me hizo preguntas sobre qué estructura de datos usar (Stack vs Queue vs PriorityQueue), qué debía guardar en cada nodo de la frontera (estado + lista de acciones acumuladas), y por qué el control de visitados debía usar solo el estado y no el par completo.

Prompts de Claude

Perfecto. Empecemos con DFS, que es el más simple. Antes de tocar código, repasemos juntos la lógica con preguntas — respóndeme con tus propias palabras (no busques la respuesta "perfecta", solo lo que recuerdes o intuyas):

Pregunta 1: En DFS necesitamos una estructura de datos para la "frontera" (los nodos que faltan por explorar). Ya viste que `algorithms/utils.py` te da `Stack`, `Queue` y `PriorityQueue`. Según lo que recuerdas de cómo funciona DFS (explorar lo más profundo posible antes de retroceder), ¿cuál de esas tres estructuras usarías y por qué?

Cuando me contestes esa, seguimos con la siguiente pieza (qué guardar en cada nodo de la frontera, cómo controlar visitados, etc.) y vas armando tu código pieza por pieza en tu editor.

Queue?

Casi, pero pensemos un poco más el porqué antes de corregir.

Una `Queue` (cola) es FIFO: **el primero que entra es el primero que sale**. Es decir, si metes A, luego B, luego C, al sacar obtienes A, B, C — en el mismo orden en que entraron. Eso favorece explorar **por niveles/anchura** (todos los vecinos "cerca" del inicio antes de ir más lejos).

DFS necesita lo contrario: quiere seguir profundizando por una rama hasta el fondo antes de retroceder a probar otra. Entonces necesita sacar **el último que metió**, no el primero.

Pregunta para ti: de las tres estructuras (`Stack`, `Queue`, `PriorityQueue`), ¿cuál sigue la regla "el último que entra es el primero que sale" (LIFO)? Fíjate en el nombre de la clase, te da una pista fuerte 😊

Stack entonces

Stack entonces

¡Exacto! `Stack` es LIFO (Last In, First Out) — perfecto para DFS.

Ahora la siguiente pieza: ¿qué guardamos dentro de cada elemento de la pila?

Piénsalo así: cuando sacas un nodo de la pila y resulta que aún no es la meta, necesitas poder seguir expandiéndolo — pero también necesitas recordar cómo llegaste hasta ahí, para poder devolver la ruta completa al final.

Pregunta 2: Si solo guardaras el estado (por ejemplo la posición `(x, y)`) en la pila, cuando llegues a la meta... ¿podrías decirle al robot qué secuencia de movimientos (NORTH, SOUTH, EAST, WEST) tiene que hacer para llegar ahí? ¿Qué otra cosa necesitarías guardar junto con el estado para no perder esa información?

los nodos que ya visito

Buena idea, pero eso responde a otra pregunta que también vamos a necesitar (cómo evitar ciclos) — llegaremos a eso en un momento.

Ahora mismo estoy preguntando algo distinto: imagina que el robot ya llegó a la meta. Para decirle "haz este camino", necesitas la lista de movimientos `([SOUTH, EAST, SOUTH, ...])` que lo llevaron ahí desde el inicio.

Si en la pila solo guardaste la posición `(x, y)` de cada nodo, al sacarlo de la pila ya no sabes qué camino de acciones tomaste para llegar a esa posición — esa información se perdió.

Pregunta 2 (reformulada): ¿Qué información adicional a la posición `(x, y)` necesitarías empujar (push) junto con cada nodo en la pila, para que cuando llegues a la meta puedas simplemente devolver esa información como la respuesta final?

Pista: piensa en qué te pide devolver la función `depthFirstSearch` según el docstring del esqueleto ("retornar una lista de acciones").

un array con los vertices?



Si guardas los **vértices/posiciones** por los que pasaste, tendrías algo como `[(1,1), (1,2), (1,3)]` — eso te dice *dónde* estuviste, pero no *qué acción* tomaste para moverte de una a otra (¿fue SOUTH? ¿EAST?). Y recuerda que `getSuccessors` te da justamente la tripleta `(nextState, action, stepCost)` — el `action` ya viene ahí, listo para guardar.

Pregunta: en vez de guardar las posiciones por las que pasaste, ¿qué pasaría si guardas en ese array las **acciones** (`Directions.SOUTH`, `Directions.EAST`, etc.) que tomaste en cada paso, desde el inicio hasta donde vas? ¿Eso no sería exactamente lo que la función tiene que devolver al final?

Con eso en mente, dime: cada elemento que empujas a la pila, ¿qué par de cosas tendría? (piensa: "estado actual" + "___")

el estado actual + la acción que hizo para llegar a ese estado

¡Muy cerca! Solo un ajuste importante.

Si guardas "estado actual + la acción que hizo para llegar ahí" (una sola acción), al final solo sabrías el **último** movimiento, no toda la secuencia desde el inicio.

Piénsalo con un ejemplo: el robot arranca en R, hace SOUTH, luego EAST, luego SOUTH otra vez. Cuando llega a la meta, necesita devolver `[SOUTH, EAST, SOUTH]` — la lista **completa** de movimientos, no solo el último.

Pregunta: en vez de guardar solo "la acción" de ese paso, ¿qué tal si guardas "la lista acumulada de todas las acciones desde el inicio hasta este estado"? Es decir, cuando expandes un nodo y generas un sucesor, tomas la lista de acciones que ya traías y le agregas la nueva acción al final.

Con esa idea: ¿cómo describirías ahora el par (o tupla) que vas a meter en la pila? Y pista extra: ¿cuál sería el elemento inicial que metes a la pila antes de empezar el ciclo `while`?

la posición inicial + una lista vacía para ir guardando las acciones

¡Exacto! Eso es. Entonces tu primer `push` sería algo como: posición inicial + `[]` (lista vacía de acciones).

Ahora sí volvamos a tu idea de antes: **los visitados**. Esto es clave porque el enunciado menciona que los mapas tienen "bifurcaciones, callejones y ciclos" — si no controlas visitados, el robot podría dar vueltas infinitas.

Pregunta 3: Piensa en el flujo general de tu `while`:

1. Sacas un nodo de la pila (`pop`).
2. ¿Es la meta? Si sí, devuelves las acciones.
3. Si no, expandes con `getSuccessors`.

¿En qué momento exacto marcarías un estado como "visitado" — cuando lo sacas de la pila (`pop`), o cuando lo metes (`push`) por primera vez? Y más importante: ¿por qué crees que importa la diferencia? (piensa en qué pasaría si el mismo estado se mete a la pila dos veces antes de que ninguna de las dos copias sea sacada)

comparo el estado y los movimientos si ya estan marcados no lo meto y si no pues los meto

Pensó durante 6s

Bien pensado — esa lógica (marcar como visitado cuando lo metes a la pila, para no volver a meterlo) funciona correctamente para DFS. La diferencia entre marcar al meter vs. al sacar en realidad importa más para UCS (te lo explico cuando lleguemos ahí, porque ahí sí puede causar errores), pero para DFS con lo que propones ya evitas los ciclos infinitos.

Ya tienes todas las piezas conceptuales:

1. Estructura: `Stack` (LIFO).
2. Cada elemento de la pila: `(estado, lista_de_acciones)`.
3. Arrancas metiendo `(estado_inicial, [])`.
4. Controlas visitados para no reprocesar estados y evitar ciclos.

A partir de esas respuestas escribí mi propia versión.

```
def depthFirstSearch(problem: SearchProblem):  
    Search the deepest nodes in the search tree first.  
  
    Your search algorithm needs to return a list of actions that reaches the  
    goal. Make sure to implement a graph search algorithm.  
  
    To get started, you might want to try some of these simple commands to  
    understand the search problem that is being passed in:  
  
    print("Start:", problem.getStartState())  
    print("Is the start a goal?", problem.isGoalState(problem.getStartState()))  
    print("Start's successors:", problem.getSuccessors(problem.getStartState()))  
    """"  
    # TODO: Add your code here  
    pila = utils.Stack  
    estado_inicial = problem.getStartState()  
    pila.push((estado_inicial, []))  
    visitados = set()  
    while not pila.isEmpty():  
        estado, acciones = pila.pop()  
        if problem.isGoalState(estado):  
            return acciones  
        if estado not in visitados:  
            visitados.add(estado)  
            for sucesor, accion, _ in problem.getSuccessors(estado):  
                pila.push((sucesor, acciones + [accion]))  
  
    return []
```

Retroalimentación de la IA: identificó que `utils.Stack` (sin paréntesis) hace referencia a la clase y no crea una instancia utilizable, por lo que `pila.push(...)` fallaría. Sugirió agregar los paréntesis: `utils.Stack()`.

```
def depthFirstSearch(problem: SearchProblem):
    """
    Search the deepest nodes in the search tree first.
    Graph search: uses a Stack (LIFO) and marks states as visited when popped.
    """
    frontier = utils.Stack()
    startState = problem.getStartState()
    frontier.push((startState, [])) # (state, actions)

    visited = set()

    while not frontier.isEmpty():
        state, actions = frontier.pop()

        if problem.isGoalState(state):
            return actions

        if state in visited:
            continue
        visited.add(state)

        for nextState, action, stepCost in problem.getSuccessors(state):
            if nextState not in visited:
                frontier.push((nextState, actions + [action]))

    return [] # No solution found
```

Cambio aceptado: corregí `pila = utils.Stack` a `pila = utils.Stack()`. El resto de la lógica (estructura de la frontera, control de visitados al hacer pop) fue diseño propio y se mantuvo sin cambios.

Verificación: ejecuté la función sobre los mapas de `layouts/diagnostic/`:

- `tinyDiagnostic`: costo total 95, 66 nodos expandidos, 0.0 s.
- `orbitalDiagnosticComplex`: costo total 196, 192 nodos expandidos, 0.0 s.

```
Ejecutando:
C:\Users\salez\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe "C:\Users\salez\OneDrive\Escritorio\EstacionEspacial 1\EstacionEspacial\main.py" -p DiagnosticProblem -f depthFirstSearch -l tinyDiagnostic -x 0.1 -z 1.0

RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 95 in 0.0 seconds
Search nodes expanded: 66
¡Misión completada! Costo: 95

Ejecución finalizada.
Presiona Enter para volver al menu...
```

```

Ejecutando:
C:\Users\salez\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe "C:\Users\salez\OneDrive\Escritorio\EstacionEspacial 1\EstacionEspacial\main.py" -p DiagnosticProblem -f depthFirstSearch -l orbitalDiagnosticComplex -x 0.25 -z 1.0

RobotPositions: [(4, 20)]
MissionSymbols: {'D': [(38, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function depthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 196 in 0.0 seconds
Search nodes expanded: 192

Ejecucion finalizada.
Presiona Enter para volver al menu...

```

Punto 2: Ruta más corta mediante BFS

Proceso de desarrollo: partiendo de mi implementación ya corregida de DFS, la IA me guio a identificar que BFS requiere una estructura FIFO en vez de LIFO, señalando la diferencia conceptual entre Stack y Queue sin darme el código. Con esa idea, adapté mi propia solución de DFS reemplazando la estructura de datos.

```

def breadthFirstSearch(problem: SearchProblem):
    """
    Search the shallowest nodes in the search tree first.
    """
    # TODO: Add your code here
    cola = utils.Queue()
    estado_inicial = problem.getStartState()
    cola.push((estado_inicial, []))
    visitados = set()
    while not cola.isEmpty():
        estado, acciones = cola.pop()
        if problem.isGoalState(estado):
            return acciones
        if estado not in visitados:
            visitados.add(estado)
            for sucesor, accion, _ in problem.getSuccessors(estado):
                cola.push((sucesor, acciones + [accion]))
    return []

```

Verificación:

- tinyDiagnostic: costo total 24, 147 nodos expandidos, 0.0 s.
- orbitalDiagnosticComplex: costo total 101, 591 nodos expandidos, 0.0 s.

```

Ejecutando:
C:\Users\salez\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe "C:\Users\salez\OneDrive\Escritorio\EstacionEspacial 1\EstacionEspacial\main.py" -p DiagnosticProblem -f breadthFirstSearch -l tinyDiagnostic -x 0.25 -z 1.0

RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 147
¡Misión completada! Costo: 24

Ejecucion finalizada.
Presiona Enter para volver al menu...

```

```

Ejecutando:
  C:\Users\salez\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe "C:\Users\salez\OneDrive\Escritorio\EstacionEspacial 1\EstacionEspacial\main.py" -p DiagnosticProblem -f breadthFirstSearch -l orbitalDiagnosticComplex -x 0.1 -z 1.0

RobotPositions: [(4, 20)]
MissionSymbols: {'D': [(38, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function breadthFirstSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 101 in 0.0 seconds
Search nodes expanded: 591
¡Misión completada! Costo: 101

Ejecucion finalizada.
Presiona Enter para volver al menu...

```

Punto 3: Navegación de menor costo mediante UCS

Proceso de desarrollo: este fue el punto más complejo, porque introduce el manejo de costos acumulados. La IA me guio con preguntas sobre qué estructura permite extraer siempre el elemento de menor prioridad (PriorityQueue), qué información adicional debía viajar junto con cada nodo (el costo acumulado, no un contador externo).

Prompts de claude:

el acumulado mas lo que cueste el sucesor siguiente

Sí, eso es correcto para el **priority** — la prioridad de un sucesor es `costoAcumulado + stepCost`.

Ahora falta la otra mitad de mi pregunta: ¿cómo quedaría la **tupla que guardas como item** dentro de la cola? Ya no sería solo `(estado, acciones)` como en DFS/BFS — necesitas un tercer valor ahí también, para que cuando saques ese nodo más adelante sepas cuál era su costo acumulado (y así poder calcular el costo de sus propios sucesores).

Pregunta: ¿qué tres cosas guardarías en esa tupla **item** ? (estado, acciones, y... ¿qué más?)

estado, acciones y el costo

Exacto. La tupla queda: `(estado, acciones, costoAcumulado)`.

Entonces el patrón completo sería:

- `frontier.push((estado, acciones, costoAcumulado), prioridad)` — donde `prioridad = costoAcumulado`.
- Al hacer `pop()`, obtienes de vuelta `(estado, acciones, costoAcumulado)`.
- Al expandir un sucesor con `stepCost`, el nuevo costo acumulado es `costoAcumulado + stepCost`, y ese mismo valor es la prioridad con la que lo empujas.

Versión final:

```
def uniformCostSearch(problem: SearchProblem):
    """
    Search the node of least total cost first.
    """

    # TODO: Add your code here
    prioridad = utils.PriorityQueue()
    estado_inicial = problem.getStartState()
    prioridad.push((estado_inicial, [], 0), 0)
    visitados = set()
    while not prioridad.isEmpty():
        estado, acciones, costo = prioridad.pop()
        if problem.isGoalState(estado):
            return acciones
        if estado not in visitados:
            visitados.add(estado)
            for sucesor, accion, costo_sucesor in problem.getSuccessors(estado):
                prioridad.push((sucesor, acciones + [accion], costo + costo_sucesor), costo + costo_sucesor)
    return []
```

Verificación:

- tinyDiagnostic: costo total 24, 134 nodos expandidos, 0.0 s.
- orbitalDiagnosticComplex: costo total 62, 560 nodos expandidos, 0.0 s.

```
Ejecutando:
C:\Users\salez\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe "C:\Users\salez\OneDrive\Escritorio\EstacionEspacial 1\EstacionEspacial\main.py" -p DiagnosticProblem -f uniformCostSearch -l tinyDiagnostic -x 0.25 -z 1.0

RobotPositions: [(2, 10)]
MissionSymbols: {'D': [(18, 2)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 134
¡Misión completada! Costo: 24

Ejecucion finalizada.
Presiona Enter para volver al menu...
```

```
Ejecutando:
C:\Users\salez\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe "C:\Users\salez\OneDrive\Escritorio\EstacionEspacial 1\EstacionEspacial\main.py" -p DiagnosticProblem -f uniformCostSearch -l orbitalDiagnosticComplex -x 0.1 -z 1.0

RobotPositions: [(4, 20)]
MissionSymbols: {'D': [(38, 4)], 'C': [], 'M': [], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type DiagnosticProblem
Misión: Alcanzar el terminal de diagnóstico D
Tipo de misión: diagnostic
Path found with total cost of 62 in 0.0 seconds
Search nodes expanded: 560
¡Misión completada! Costo: 62

Ejecucion finalizada.
Presiona Enter para volver al menu...
```

Punto 4: UCS con módulo obligatorio y costo dependiente del estado

a) Complete isGoalState en ModuleRepairProblem.

```
def isGoalState(self, state):
    """
    Returns True if the robot reached C after picking up M.
    """
    position, hasModule = state

    if position == self.controlPosition and hasModule == True:
        return True

    return False
```

Para completar `isGoalState` se trabajó con el estado, separando la posición del robot y si ya había recogido el módulo. Así se definió como estado objetivo aquel en el que el robot tiene el módulo y se encuentra en el centro de control.

b) Complete `_getStepCost` en `ModuleRepairProblem`.

```
def _getStepCost(self, nextPosition, hasModule):
    """
    Returns the movement cost for entering nextPosition.
    """
    x, y = nextPosition
    costo = self.startingMissionState.getTerrainCost(x, y)

    if hasModule == True:
        costo = costo * 2

    return costo
```

Para completar `_getStepCost` se tomó la siguiente posición del robot y se obtuvo el costo normal del terreno. Luego se revisó el estado para saber si el robot ya tenía el módulo. Si ya lo tenía, el costo del movimiento se multiplicaba por dos de acuerdo con las reglas de la misión.

c) Complete `getSuccessors` en `ModuleRepairProblem` respetando el formato (`nextState`, `direction`, `stepCost`).


```

def getSuccessors(self, state):
    successors = []
    self._expanded += 1
    position, hasModule = state
    x, y = position

    for direction in [
        Directions.NORTH,
        Directions.SOUTH,
        Directions.EAST,
        Directions.WEST,
    ]:
        dx, dy = Actions.directionToVector(direction)

        nextx = int(x + dx)
        nexty = int(y + dy)

        if not self.walls[nextx][nexty]:
            nextPosition = (nextx, nexty)

```

```

            nextPosition = (nextx, nexty)

            costo = self._getStepCost(nextPosition, hasModule)

            nextHasModule = hasModule

            if nextPosition == self.modulePosition:
                nextHasModule = True
                (variable) nextPosition: tuple[int, int]
            nextState = (nextPosition, nextHasModule)

            successors.append(
                (nextState, direction, costo)
            )

    return successors

```

Para completar `getSuccessors` se tomó el estado actual del robot y se generaron los posibles movimientos en las cuatro direcciones. Para cada movimiento se calculó la nueva posición y se verificó que no fuera un muro. Luego se construyó el siguiente estado teniendo en cuenta si el robot recogía el módulo y se agregó junto con la dirección y el costo del movimiento.

d) Para esta misión, utilice la clase `ModuleRepairProblem` y resuelva los mapas disponibles en `layouts/module` mediante `uniformCostSearch`, implementado en el punto 3.

Para completar este punto se utilizó ModuleRepairProblem junto con uniformCostSearch para ejecutar todos los mapas disponibles en layouts/module. En cada caso se verificó que el robot completara la misión correctamente recogiendo primero el módulo y llegando después al centro de control.

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem -f uniformCostSearch -l tinyModuleRun
RobotPositions: [(1, 3)]
MissionSymbols: {'D': [], 'C': [(13, 4)], 'M': [(6, 8)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 48 in 0.0 seconds
Search nodes expanded: 138
¡Misión completada! Costo: 48
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem -f uniformCostSearch -l moduleVault
RobotPositions: [(15, 11)]
MissionSymbols: {'D': [], 'C': [(15, 3)], 'M': [(5, 6)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 80 in 0.0 seconds
Search nodes expanded: 245
¡Misión completada! Costo: 80
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem -f uniformCostSearch -l reactorSupplyBay
RobotPositions: [(3, 5)]
MissionSymbols: {'D': [], 'C': [(21, 5)], 'M': [(18, 14)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 85 in 0.0 seconds
Search nodes expanded: 310
¡Misión completada! Costo: 85
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem -f uniformCostSearch -l orbitalCargoMaze
RobotPositions: [(5, 10)]
MissionSymbols: {'D': [], 'C': [(27, 4)], 'M': [(29, 19)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 167 in 0.0 seconds
Search nodes expanded: 797
¡Misión completada! Costo: 167
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem
-f uniformCostSearch -l cargoLoopDeck
RobotPositions: [(1, 12)]
MissionSymbols: {'D': [], 'C': [(6, 3)], 'M': [(17, 11)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 136 in 0.0 seconds
Search nodes expanded: 285
¡Misión completada! Costo: 136
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem
-f uniformCostSearch -l controlWingCargo
RobotPositions: [(19, 4)]
MissionSymbols: {'D': [], 'C': [(3, 5)], 'M': [(7, 12)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 74 in 0.0 seconds
Search nodes expanded: 290
¡Misión completada! Costo: 74
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem
-f uniformCostSearch -l floodedServiceDeck
RobotPositions: [(23, 13)]
MissionSymbols: {'D': [], 'C': [(14, 8)], 'M': [(5, 4)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 99 in 0.0 seconds
Search nodes expanded: 372
¡Misión completada! Costo: 99
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem
-f uniformCostSearch -l loadingSpine
RobotPositions: [(27, 5)]
MissionSymbols: {'D': [], 'C': [(21, 17)], 'M': [(4, 11)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 123 in 0.0 seconds
Search nodes expanded: 632
¡Misión completada! Costo: 123
```

```
C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem
-f uniformCostSearch -l moduleRelayRooms
RobotPositions: [(3, 15)]
MissionSymbols: {'D': [], 'C': [(5, 4)], 'M': [(23, 7)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 159 in 0.0 seconds
Search nodes expanded: 525
¡Misión completada! Costo: 159
```

```

C:\Users\MParra14\Documents\IA\Taller-1-IA\EstacionEspacial>python main.py -p ModuleRepairProblem
-f uniformCostSearch -l stationModuleComplex
RobotPositions: [(3, 21)]
MissionSymbols: {'D': [], 'C': [(33, 17)], 'M': [(19, 5)], 'K': [], 'T': []}
[SearchAgent] using function uniformCostSearch
[SearchAgent] using problem type ModuleRepairProblem
Misión: Recoger M y entregarlo en C
Tipo de misión: module
Path found with total cost of 208 in 0.0 seconds
Search nodes expanded: 1122
¡Misión completada! Costo: 208

```

En la siguiente tabla se resumen los resultados obtenidos al ejecutar **uniformCostSearch** con **ModuleRepairProblem** en cada uno de los mapas disponibles. Se muestran el costo total de la solución encontrada y la cantidad de nodos expandidos en cada caso.

Layout	Costo	Nodos Expandidos
tinyModuleRun	48	138
moduleVault	80	245
reactorSupplyBay	85	310
orbitalCargoMaze	167	797
cargoLoopDeck	136	285
controlWingCargo	74	290
floodedServiceDeck	99	372
loadingSpine	123	632
moduleRelayRooms	159	525
stationModuleComplex	208	1122

Verificación con IA

Después de realizar la implementación, se le pidió a la IA que revisara posibles errores o casos que no se estuvieran teniendo en cuenta. La IA señaló que en `getSuccessors` no se valida de forma explícita que la siguiente posición esté dentro de los límites del mapa antes de acceder a `self.walls`. Esto podría ser un problema en algunos casos por la indexación negativa de Python.

Prompt: Dime cómo ves la implementación actual y posibles mejoras o consideraciones:

`isGoalState` — Correcto y simple, verifica ambas condiciones (posición y módulo).

`_getStepCost` + orden de evaluación en `getSuccessors` — Este es el punto más delicado del enunciado y lo resolviste bien: calculas `costo` usando el `hasModule` del estado de origen (antes de actualizarlo), y después actualizas `nextHasModule`. Eso hace que:

- El paso que entra a M use `hasModule=False` → costo normal ✓
- Los pasos siguientes usen `hasModule=True` → costo doble ✓

Esto es exactamente lo que pide el enunciado ("el movimiento que entra a M conserva costo normal, a partir del siguiente se duplica"). Muchos grupos se equivocan aquí duplicando el costo del paso que entra a M — tú no.

`getCostOfActions` — Consistente con la misma lógica (usa `hasModule` antes de actualizarlo, luego actualiza). Bien.

⚠ Un detalle a revisar (edge case, no necesariamente un bug en tu proyecto)

python

```
if not self.walls[nextx][nexty]:
```

No validas explícitamente que `nextx, nexty` estén dentro de los límites del mapa antes de indexar. En Python, si `nextx = -1` (por ejemplo, el robot en $x=0$ y se mueve hacia la izquierda), `self.walls[-1]` no lanza error — accede a la última fila del arreglo por indexación negativa. Si el layout no tiene una fila de muros perimetrales completa, esto podría generar vecinos "fantasma" inválidos silenciosamente (sin excepción, sin aviso).

Sin embargo, se decidió no agregar esta validación porque los mapas del proyecto cuentan con muros en los bordes y la solución funcionó correctamente en todos los layouts probados. Por esta razón, se mantuvo la implementación original al considerarse suficiente para las condiciones del taller.

Punto 5: A* y diseño de heurísticas

Punto A

Proceso de desarrollo: Para este punto le pedí a la IA que verificara lo que ya se había hecho y lo que faltaba por completar. En este caso me dio por completo el código de A* así que yo hice una versión más simplificada y que tuviera la misma lógica.

Prompt de Chat GPT: Hola bro, podrias verificar este pdf y el zip, y decirme que falta?

Respuesta para este punto:

A*

Este sí te faltaba completamente.

Reemplaza:

 Python



 Ejecutar

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):  
    ...
```

por:

 Python



 Ejecutar

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):  
    """  
    Search the node that has the lowest combined cost and heuristic first.  
    """  
  
    frontier = utils.PriorityQueue()  
    start = problem.getStartState()  
  
    frontier.push(  
        (start, [], 0),  
        heuristic(start, problem)  
    )  
  
    bestCost = {start: 0}  
  
    while not frontier.isEmpty():  
  
        state, actions, cost = frontier.pop()  
  
        if cost > bestCost.get(state, float("inf")):  
            continue  
  
        if problem.isGoalState(state):  
            return actions  
  
        for successor, action, stepCost in problem.getSuccessors(state):  
  
            newCost = cost + stepCost  
  
            if newCost < bestCost.get(successor, float("inf")):  
  
                bestCost[successor] = newCost  
  
                priority = newCost + heuristic(succes↓, problem)
```

 Pregúntale a ChatGPT

Alta ▾



```
        frontier.push(
            (successor, actions + [action], newCost),
            priority
        )

    return []
```

La línea más importante de A* es:

 Python



 Ejecutar

```
priority = newCost + heuristic(successor, problem)
```

Es decir:

$f(n) = g(n) + h(n)$



$g(n)$ = costo que ya llevamos.

$h(n)$ = estimación de lo que falta.

Versión final:

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):
    """
    Search the node that has the lowest combined cost and heuristic first.
    """
    # cola de prioridad para ir sacando el mejor nodo cada vez
    cola = utils.PriorityQueue()

    inicio = problem.getStartState()
    # metemos el inicio: (estado, camino recorrido, costo acumulado)
    cola.push((inicio, [], 0), heuristic(inicio, problem))

    # aquí guardamos el mejor costo que hemos encontrado para cada estado
    costos = {inicio: 0}

    while not cola.isEmpty():
        estado, camino, costo = cola.pop()

        # si ya encontramos algo mejor para este estado, lo saltamos
        if costo > costos.get(estado, float("inf")):
            continue

        # si llegamos a la meta, ya ganamos
        if problem.isGoalState(estado):
            return camino

        # revisamos a donde nos podemos mover desde aquí
        for sucesor, accion, costoPaso in problem.getSuccessors(estado):
            nuevoCosto = costo + costoPaso

            # solo nos interesa si es mejor que lo que ya teníamos
            if nuevoCosto < costos.get(sucesor, float("inf")):
                costos[sucesor] = nuevoCosto
                prioridad = nuevoCosto + heuristic(sucesor, problem)
                cola.push((sucesor, camino + [accion], nuevoCosto), prioridad)

    # si no encontramos nada, pues no hay camino
    return []
```

Verificación:

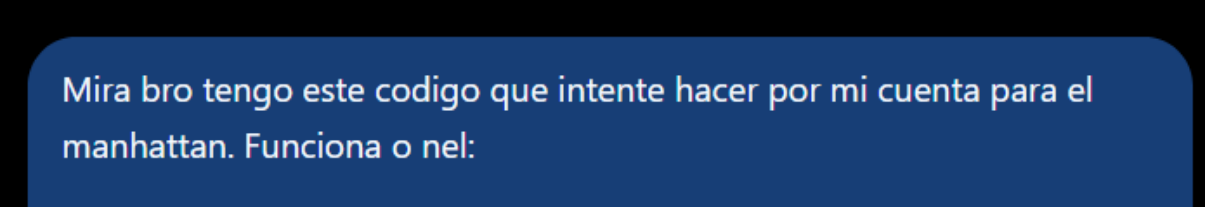
- null Heuristic: Costo 24, 148 nodos expandidos, 0.0s

```
RobotPositions: [(1, 7)]
MissionSymbols: {'D': [], 'C': [(9, 7)], 'M': [], 'K': [(5, 3)], 'T': [(11, 1)]}
[SearchAgent] using function aStarSearch and heuristic nullHeuristic
[SearchAgent] using problem type SystemRepairProblem
Misión: Recoger K, reparar todos los T y volver a C
Tipo de misión: systems
Path found with total cost of 24 in 0.0 seconds
Search nodes expanded: 148
¡Misión completada! Costo: 24
```


Punto B

Proceso de desarrollo: Para este punto intente hacer la heurística por mi cuenta teniendo en cuenta algunos consejos que dan en internet. También le pedí consejo a chat, para que verificará si estaba bien.

Prompt de chat:



Mira bro tengo este codigo que intente hacer por mi cuenta para el manhattan. Funciona o no:

Respuesta para este punto:

Sí bro, ese Manhattan está bien . De hecho, la lógica que hiciste coincide con lo que pide el taller para la heurística base: ir hacia **K** si todavía no tienes el kit, hacia el **T** pendiente más cercano si ya tienes el kit, y hacia **C** cuando ya no quedan sistemas por reparar.  Pdf

Tu lógica es esta:

 Python

```
if not hasKit:
```

👉 No tiene **K** → calcula Manhattan hasta **K**.

Luego:

 Python

```
if len(pendingSystems) > 0:
```

👉 Ya tiene **K**, pero quedan **T** → calcula la distancia a todos los **T** pendientes y devuelve:

 Python

```
min(distancias)
```

o sea, el más cercano.

Y finalmente:

 Python

```
x2, y2 = problem.controlPosition  
return abs(x1 - x2) + abs(y1 - y2)
```

👉 Ya no quedan **T** → estima la distancia hasta **C**.

Tu fórmula también está correcta

 Python

```
abs(x1 - x2) + abs(y1 - y2)
```

es exactamente distancia Manhattan:

$$|x_1 - x_2| + |y_1 - y_2|$$

Y hay algo bueno de tu implementación

No estás teniendo en cuenta paredes ni terrenos costosos directamente. Para una heurística eso está bien, porque Manhattan está dando una estimación optimista de la distancia mínima. Las paredes o los terrenos más costosos pueden hacer que el camino real cueste más, pero Manhattan no debería sobreestimarlos.

Por eso sirve como heurística admisible para este problema.

Versión final:

```
def manhattanHeuristic(state, problem):  
    """  
    The Manhattan distance heuristic.  
  
    Baseline rule for this workshop: estimate the direct distance to the next  
    mandatory target:  
    - K if the robot does not have the kit yet.  
    - the nearest pending T if the robot has the kit and systems remain.  
    - C if all systems have been repaired.  
    """  
    position, hasKit, pendingSystems = state  
    x1, y1 = position  
  
    # si todavia no tiene el kit, el objetivo mas cercano es ir por el kit  
    if not hasKit:  
        x2, y2 = problem.kitPosition  
        return abs(x1 - x2) + abs(y1 - y2)  
  
    # si ya tiene el kit pero faltan sistemas por reparar,  
    # calculamos la distancia al T mas cercano  
    if len(pendingSystems) > 0:  
        distancias = []  
        for sistema in pendingSystems:  
            x2, y2 = sistema  
            distancias.append(abs(x1 - x2) + abs(y1 - y2))  
        return min(distancias)  
  
    # si ya reparo todo, lo unico que falta es volver al centro de control  
    x2, y2 = problem.controlPosition  
    return abs(x1 - x2) + abs(y1 - y2)
```

Verificación:

- manhattan Heuristic: Costo 24, 127 nodos expandidos, 0.0s

```
RobotPositions: [(1, 7)]  
MissionSymbols: {'D': [], 'C': [(9, 7)], 'M': [], 'K': [(5, 3)], 'T': [(11, 1)]}  
[SearchAgent] using function aStarSearch and heuristic manhattanHeuristic  
[SearchAgent] using problem type SystemRepairProblem  
Misión: Recoger K, reparar todos los T y volver a C  
Tipo de misión: systems  
Path found with total cost of 24 in 0.0 seconds  
Search nodes expanded: 127  
¡Misión completada! Costo: 24
```

Punto C

Proceso de desarrollo: Utilice la misma metodología del punto B

Prompt de chat:

Listo bro, avancemos. Tengo este para el euclidean.

Respuesta para este punto:

Sí bro, ese `euclideanHeuristic` está bien  y está alineado con la regla que pide el taller para este punto: ir hacia `K` si aún no tienes el kit, hacia el `T` pendiente más cercano si ya lo tienes, y finalmente hacia `C` cuando no quedan sistemas.  Pdf

Tu función hace correctamente estas tres cosas:

`</>` Python



 Ejecutar

```
if not hasKit:
```

→ calcula distancia en línea recta hasta `K`.

Luego:

`</>` Python



 Ejecutar

```
if len(pendingSystems) > 0:
```

→ calcula la distancia Euclidiana a todos los `T` pendientes y devuelve la menor.

Y al final:

`</>` Python



 Ejecutar

```
x2, y2 = problem.controlPosition
```

→ cuando ya no quedan `T`, estima la distancia hasta `C`.

La fórmula también está correcta:

`</>` Python



 Ejecutar

```
((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5
```

que es:

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Versión final:

```
def euclideanHeuristic(state, problem):  
    """  
    The Euclidean distance heuristic.  
  
    Baseline rule for this workshop: estimate the direct distance to the next  
    mandatory target:  
    - K if the robot does not have the kit yet.  
    - the nearest pending T if the robot has the kit and systems remain.  
    - C if all systems have been repaired.  
    """  
    position, hasKit, pendingSystems = state  
    x1, y1 = position  
  
    # si no tiene el kit, calculamos distancia en linea recta hasta K  
    if not hasKit:  
        x2, y2 = problem.kitPosition  
        return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5  
  
    # si ya tiene el kit pero faltan T's, buscamos el mas cercano  
    if len(pendingSystems) > 0:  
        distancias = []  
        for sistema in pendingSystems:  
            x2, y2 = sistema  
            distancias.append(((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5)  
        return min(distancias)  
  
    # si ya reparo todo, falta volver a C  
    x2, y2 = problem.controlPosition  
    return ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5
```

Verificación:

- euclidean Heuristic: Costo 24, 129 nodos expandidos, 0.0s

```
RobotPositions: [(1, 7)]  
MissionSymbols: {'D': [], 'C': [(9, 7)], 'M': [], 'K': [(5, 3)], 'T': [(11, 1)]}  
[SearchAgent] using function aStarSearch and heuristic euclideanHeuristic  
[SearchAgent] using problem type SystemRepairProblem  
Misión: Recoger K, reparar todos los T y volver a C  
Tipo de misión: systems  
Path found with total cost of 24 in 0.0 seconds  
Search nodes expanded: 129  
¡Misión completada! Costo: 24
```

Punto D

Proceso de desarrollo: Utilice la misma metodología de los puntos anteriores

Prompt de chat:

Listo bro. Para el siguiente intento hacerlo por mi cuenta otra vez y me dio lo siguiente:

Respuesta para este punto:

Bro, vas bien y tu idea sí tiene sentido. De hecho, tu heurística actual es admisible, pero tiene un detalle importante: **no siempre es consistente**. Y como el taller dice que debe ser admisible y *preferiblemente consistente*, yo le haría un ajuste pequeño en lugar de botarla y empezar de cero. Pdf

Tu idea actual es:

`</> Python`



Ejecutar

```
si no tengo K:  
    distancia actual → K  
    +  
    distancia K → T más lejano  
  
si tengo K y quedan T:  
    distancia → T más lejano  
  
si no quedan T:  
    distancia → C
```

Eso es una buena base. El problema aparece justo cuando reparas el último **T**.

Por ejemplo, imagina:

```
Robot → T = 1 paso  
T → C = 10 pasos
```



Antes de reparar el último **T**, tu heurística dice:

```
h(s) = 1
```



Das un paso, reparas **T**, y ahora como `pendingSystems` queda vacío, tu heurística pasa a:

```
h(siguiete) = 10
```



Entonces tendríamos algo así:

```
1 <= costoPaso + 10
```



y la heurística **aumenta bruscamente después de avanzar hacia la solución**. Esa transición es la que nos rompe la consistencia.

La modificación es pequeña

En lugar de mirar únicamente:

```
</> Python  
max(distancias)
```



Ejecutar

cuando quedan **T**, incluye también a **C** como uno de los lugares que obligatoriamente tendrás que alcanzar.

Es decir, sobre **tu código**, podrías modificar esta parte:

```
</> Python  
if len(pendingSystems) > 0:  
  
    # de todas formas hay que llegar hasta el T mas lejano  
    distancias = [manhattan(actual, sistema) for sistema in pendingSystems]  
  
    total += max(distancias)
```



Ejecutar

para que contemple también la distancia a **C**:

```
</> Python  
if len(pendingSystems) > 0:  
  
    # debemos visitar todos los T y eventualmente regresar a C  
    distancias = [manhattan(actual, sistema) for sistema in pendingSystems]  
  
    distanciaControl = manhattan(actual, problem.controlPosition)  
  
    total += max(max(distancias), distanciaControl)
```



Ejecutar

Pero hay otro detallito: cuando **todavía no tienes el kit**, después de hacer:

```
</> Python  
actual = problem.kitPosition
```



Ejecutar

también queremos aplicar exactamente esa misma idea.

Versión inicial:

```
def systemRepairHeuristic(
    state: Tuple[Tuple, bool, Tuple], problem: SystemRepairProblem
):
    """
    Your heuristic for the SystemRepairProblem.

    state: (position, hasKit, pendingSystems)
    problem: SystemRepairProblem instance

    This must be admissible and preferably consistent.

    Hints:
    - Use problem.heuristicInfo to cache expensive computations
    - Go with some simple heuristics first, then build up to more complex ones
    - Consider the kit, pending systems, and the final return to control center
    - Balance heuristic strength vs. computation time (do experiments!)
    """
    position, hasKit, pendingSystems = state

    def manhattan(a, b):
        return abs(a[0] - b[0]) + abs(a[1] - b[1])

    total = 0
    actual = position

    # si no tiene el kit, primero toca pasar por K
    if not hasKit:
        total += manhattan(actual, problem.kitPosition)
        actual = problem.kitPosition

    if len(pendingSystems) > 0:
        # de todas formas hay que llegar hasta el T mas lejano
        distancias = [manhattan(actual, sistema) for sistema in pendingSystems]
        total += max(distancias)
    else:
        # si ya repara todo, falta volver a C
        total += manhattan(actual, problem.controlPosition)
```


Versión final:

```
def systemRepairHeuristic(  
    - Use problem.heuristicInfo to cache expensive computations  
    - Go with some simple heuristics first, then build up to more complex ones  
    - Consider the kit, pending systems, and the final return to control center  
    - Balance heuristic strength vs. computation time (do experiments!)  
    ""  
  
    position, hasKit, pendingSystems = state  
    def manhattan(a, b):  
        return abs(a[0] - b[0]) + abs(a[1] - b[1])  
    total = 0  
    actual = position  
    # si no tiene el kit, primero obligatoriamente debe llegar a K  
    if not hasKit:  
        total += manhattan(actual, problem.kitPosition)  
        actual = problem.kitPosition  
    if len(pendingSystems) > 0:  
        # faltan sistemas obligatorios por visitar  
        distancias = [  
            manhattan(actual, sistema)  
            for sistema in pendingSystems  
        ]  
        # tambien sabemos que eventualmente debemos terminar en C  
        distanciaControl = manhattan(  
            actual,  
            problem.controlPosition  
        )  
        total += max(  
            max(distancias),  
            distanciaControl  
        )  
    else:  
        # ya reparo todos los sistemas, solo falta C  
        total += manhattan(  
            actual,  
            problem.controlPosition  
        )  
    return total
```

Verificación:

- system repair Heuristic: Costo 24, 103 nodos expandidos, 0.0s

```
RobotPositions: [(1, 7)]  
MissionSymbols: {'D': [], 'C': [(9, 7)], 'M': [], 'K': [(5, 3)], 'T': [(11, 1)]}  
[SearchAgent] using function aStarSearch and heuristic systemRepairHeuristic  
[SearchAgent] using problem type SystemRepairProblem  
Misión: Recoger K, reparar todos los T y volver a C  
Tipo de misión: systems  
Path found with total cost of 24 in 0.0 seconds  
Search nodes expanded: 103  
¡Misión completada! Costo: 24
```

Punto E

Proceso de desarrollo: Para este punto ejecuté las pruebas necesarias, y le pedí ayuda a chat para que las agrupara en un tabla y pudiera poner los resultados ordenados.

Tabla de resultados:

Layout	Cantidad de sistemas T	Heurísticas evaluadas
tinyRepair	1	Null, Manhattan, Euclidean, SystemRepair
dualArray	2	Null, Manhattan, Euclidean, SystemRepair
triadCorridor	3	Null, Manhattan, Euclidean, SystemRepair
fourCornerArray	4	Null, Manhattan, Euclidean, SystemRepair
sixBayDeck	6	Null, Manhattan, Euclidean, SystemRepair

Punto F

Proceso de desarrollo: Use la misma metodología del punto anterior, para que fuera más fácil, el mostrar los datos en una tabla.

Tabla de resultados:

Layout	Sistemas T	Heurística	Costo de solución	Nodos expandidos	Tiempo
tinyRepair	1	Null	24	148	0.0 s
tinyRepair	1	Manhattan	24	127	0.0 s
tinyRepair	1	Euclidean	24	129	0.0 s
tinyRepair	1	SystemRepair	24	103	0.0 s
dualArray	2	Null	50	284	0.0 s
dualArray	2	Manhattan	50	263	0.0 s
dualArray	2	Euclidean	50	263	0.0 s

dualArray	2	SystemRepair	50	256	0.0 s
triadCorridor	3	Null	68	817	0.0 s
triadCorridor	3	Manhattan	68	705	0.0 s
triadCorridor	3	Euclidean	68	734	0.0 s
triadCorridor	3	SystemRepair	68	687	0.0 s
fourCornerArray	4	Null	78	2305	0.0 s
fourCornerArray	4	Manhattan	78	1972	0.0 s
fourCornerArray	4	Euclidean	78	2053	0.0 s
fourCornerArray	4	SystemRepair	78	1731	0.0 s
sixBayDeck	6	Null	114	14473	0.0 s
sixBayDeck	6	Manhattan	114	13220	0.0 s
sixBayDeck	6	Euclidean	114	13472	0.1 s
sixBayDeck	6	SystemRepair	114	10698	0.0 s

Prompt utilizado en el punto E y F:

Perfect me puedes dar las tablas para pegar en word :)

Respuesta:

Copia solo este bloque:

```
Layout  Cantidad de sistemas T  Heurísticas evaluadas
tinyRepair  1  Null, Manhattan, Euclidean, SystemRepair
dualArray  2  Null, Manhattan, Euclidean, SystemRepair
triadCorridor  3  Null, Manhattan, Euclidean, SystemRepair
fourCornerArray  4  Null, Manhattan, Euclidean, SystemRepair
sixBayDeck  6  Null, Manhattan, Euclidean, SystemRepair
```

Claro bro. Te la dejo en formato con **tabulaciones** para que primero la pegues en Excel Online y luego la pases a Word Online sin que se descuadre.

```
Layout  Sistemas T  Heurística  Costo de solución  Nodos expandidos  Tiempo
tinyRepair  1  Null      24  148  0.0 s
tinyRepair  1  Manhattan 24  127  0.0 s
tinyRepair  1  Euclidean 24  129  0.0 s
tinyRepair  1  SystemRepair 24  103  0.0 s
dualArray  2  Null      50  284  0.0 s
dualArray  2  Manhattan 50  263  0.0 s
dualArray  2  Euclidean 50  263  0.0 s
dualArray  2  SystemRepair 50  256  0.0 s
triadCorridor  3  Null      68  817  0.0 s
triadCorridor  3  Manhattan 68  705  0.0 s
triadCorridor  3  Euclidean 68  734  0.0 s
triadCorridor  3  SystemRepair 68  687  0.0 s
fourCornerArray  4  Null      78  2305  0.0 s
fourCornerArray  4  Manhattan 78  1972  0.0 s
fourCornerArray  4  Euclidean 78  2053  0.0 s
fourCornerArray  4  SystemRepair 78  1731  0.0 s
sixBayDeck  6  Null     114 14473  0.0 s
sixBayDeck  6  Manhattan 114 13220  0.0 s
sixBayDeck  6  Euclidean 114 13472  0.1 s
sixBayDeck  6  SystemRepair 114 10698  0.0 s
```

Punto 6: Reflexión sobre el trabajo realizado

Complejidad temporal y espacial de cada algoritmo

DFS: $O(b^m)$ tiempo, $O(b \cdot m)$ espacio

BFS: $O(b^d)$ tiempo y espacio

UCS: $O(b^{(1+LC^*/\epsilon J)})$ tiempo y espacio

A*: mismo peor caso que UCS, pero en la práctica mucho menor con buena heurística

Complejitud

DFS: no se completa en espacios infinitos, pero en los mapas finitos, con control de visitados sí terminó y encontró solución.

BFS, UCS, A*: completos si b es finito y los costos son ≥ 0 (o $>$ algún ϵ).

Condiciones de optimalidad

DFS: **no óptimo** — encontró costo 95 en `tinyDiagnostic` mientras BFS/UCS encontraron 24.

BFS: es óptimo, si todos los costos son iguales

UCS: óptimo siempre que costos ≥ 0

A*: óptimo si la heurística es admisible, ya que en este caso las heurísticas devolvieron el mismo costo de solución el cual fue 24, 50, 68, 78, 114, de esta forma determinamos que es admisible

Explicación de porque La posición del robot no es suficiente para representar el problema del punto 4:

La meta en el punto 4, depende de una condición que no se puede saber solo por dónde está el robot: si ya recogió el módulo M o no. Si el estado fuera solo `position`, el robot podría estar en la celda de C en dos situaciones muy distintas (con módulo o sin él), pero para el algoritmo de búsqueda ambas se verían como *exactamente el mismo estado*. Esto genera dos problemas:

1. **La meta se marcaría mal.** El algoritmo podría declarar la misión completada apenas el robot pase por C, sin haber recogido M, porque no tiene forma de distinguir ese caso.
2. **El costo se calcularía mal.** La regla del punto 4 dice que, después de recoger M, cada movimiento cuesta el doble. Si el estado no registra si ya se tiene el módulo, UCS no puede saber cuándo aplicar ese costo duplicado, y terminaría dando el mismo costo a rutas que en realidad son distintas.

Consistencia de heurísticas

Teniendo en cuenta la siguiente definición: Una heurística es consistente si nunca "promete más de lo que un paso realmente vale": $h(n) \leq \text{costo del paso} + h(n')$. En otras palabras, la estimación no puede bajar más de lo que el robot avanza realmente. Entonces podemos decir que fueron constituyentes por lo siguiente:

- **Manhattan:** mide distancia en cuadrícula ($|\Delta x| + |\Delta y|$). Cada movimiento cambia una coordenada en 1, así que h baja como máximo 1 por paso. Como el costo mínimo de un movimiento es 1, la estimación nunca "adelanta" más de lo real es consistente.
- **Euclidean:** mide distancia en línea recta. Por geometría, esta distancia tampoco puede bajar más de lo que el robot se mueve, así que también es consistente. Pero al ser una línea recta, subestima un poco más que Manhattan (la diagonal siempre es más corta que ir por la cuadrícula), por eso expandió un poco más nodos en los resultados (ej. 13,472 vs 13,220 en `sixBayDeck`).

Comparación de heurísticas

En todos los mapas probados, las cuatro heurísticas llegaron **al mismo costo de solución óptimo** (24, 50, 68, 78 y 114 según el mapa), lo cual confirma que las cuatro son admisibles: ninguna sacrificó optimalidad por velocidad.

Donde sí hay diferencias claras es en **nodos expandidos**: a medida que el mapa se complica (más sistemas T), la brecha entre heurísticas crece.

Mapa (sistemas T)	Null	Manhattan	Euclidean	SystemRepair
tinyRepair (1)	148	127	129	103
dualArray (2)	284	263	263	256
triadCorridor (3)	817	705	734	687
fourCornerArray (4)	2,305	1,972	2,053	1,731
sixBayDeck (6)	14,473	13,220	13,472	10,698

Prompt utilizado en el punto 6a:

Ayudame a rendartar, este texto para que sea mas compresible

+

Sonnet 5 Medio ▾

haz un tabla con los valores ya dados que me evidencia los diferencia entre los algoritmos Manhattan, Euclidean, Systeam Repair

+

Sonnet 5 Medio ▾

b. Reflexión sobre el uso de inteligencia artificial

A lo largo del taller nos sostuvimos con la ia para la corrección de error y doptimizacion de nuestro codigo asi que como lampara la redacción de este texto, por ejemplo
Punto 1 (DFS): la corrección fue básica, un error de sintaxis (`utils.Stack` sin paréntesis).
No tocó tu lógica de diseño, solo un detalle de implementación.

Así como, en el Punto 4: La IA señaló una mejora secundaria (falta de validación de límites del mapa), aunque se decidio no aplicarla porque no era necesaria para los mapas del proyecto. Aqui podemos evidenciar que no se aceptasto todo lo que sugirió la IA, sino que evaluo segun si aplicaba o no en este caso.

Adicionalmente en el Punto 5A en donde la IA genero codigo con las indicaciones dadas en A*, no una corrección menor. En este punto, podemos decir que la ia puede ayudar significativamente a terminar trabajos largos y complejos. pero aun asi es un herramienta la cual te puede ayudar y sin embargo estorbar en tu aprendizaje si solo la usas para saber respuestas sin analizar sobre estas.

Prompt utilizado en el punto 6b:

teniendo encuentra lo que ya hemos visto el siguiente texto redactalo mejor y añade un ejemplo donde la ia nos ayudo en nuestro trabajo



Sonnet 5 Medio ▾

